

Pandas

Pandas is an open source python library providing high performance and data manipulation and analysis tool.

```
In [1]: import pandas as pd
```

series

It is a one dimensional labelled array capable of strong / holding homogeneous data of any type (int,str,float, _ _ _ _ , python objects etc)

Pandas series is nothing but a column value in excel-sheet

Pandas series values are mutable

creating a series object

Syntax :- varname = pandas.Series(object,index,dtype)

```
In [2]: # creating series object by int type
a = 100
var = pd.Series(a)
print(var,type(var))
```

```
0    100
dtype: int64 <class 'pandas.core.series.Series'>
```

```
In [3]: # creating series object by float type
a = 100.89
var = pd.Series(a)
print(var,type(var))
```

```
0    100.89
dtype: float64 <class 'pandas.core.series.Series'>
```

```
In [4]: # creating series object by bool type
a = True
var = pd.Series(a)
print(var,type(var))
```

```
0    True
dtype: bool <class 'pandas.core.series.Series'>
```

```
In [5]: # creating series object by complex type
a = 2+3j
var = pd.Series(a)
print(var,type(var))
```

```
0    2.0+3.0j
dtype: complex128 <class 'pandas.core.series.Series'>
```

```
In [6]: # creating series object by str type
a = "Pradeep"
var = pd.Series(a)
print(var,type(var))
```

```
0    Pradeep
dtype: object <class 'pandas.core.series.Series'>
```

```
In [7]: # creating series object by bytes type
a = bytes(100)
var = pd.Series(a)
print(var,type(var))
```

```
0    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00...'
dtype: object <class 'pandas.core.series.Series'>
```

```
In [8]: # creating series object by bytearray type
a = bytearray([10,20,30])
print(a,type(a))
var = pd.Series(a)
print(var,type(var))
```

```
bytearray(b'\n\x14\x1e') <class 'bytearray'>
0    10
1    20
2    30
dtype: int64 <class 'pandas.core.series.Series'>
```

```
In [9]: # creating series object by range type
a = range(10,20,2)
var = pd.Series(a)
print(var,type(var))
```

```
0    10
1    12
2    14
3    16
4    18
dtype: int64 <class 'pandas.core.series.Series'>
```

```
In [10]: # creating series object by list type
a =[10,20,30,40,50,'pradeep']
var = pd.Series(a)
print(var,type(var))
```

```
0      10
1      20
2      30
3      40
4      50
5    pradeep
dtype: object <class 'pandas.core.series.Series'>
```

```
In [11]: # creating series object by tuple type
a = (10,20,30,40,50,'pradeep')
var = pd.Series(a)
print(var,type(var))
```

```

0      10
1      20
2      30
3      40
4      50
5  pradeep
dtype: object <class 'pandas.core.series.Series'>

```

In [12]: *# Note :- set and frozenset does not maintain insertion order so we can not conv*

creating Series object from dict

If we use dict object in Series() then keys can be taken as indices(or indexes) automatically and corresponding values of dict can be taken as Series data

```

In [13]: d1 = {"ID1":23,"ID2":34,"ID3":56,"ID4":78}
var = pd.Series(d1)
print(var,type(var))

```

```

ID1    23
ID2    34
ID3    56
ID4    78
dtype: int64 <class 'pandas.core.series.Series'>

```

```

In [14]: print(var['ID1'])

```

23

Index

NOTE-1: When we assign Integer Programmer Indices Then PVM Will not consider Default Indices

We get key when access with default indices

Note-2: When we assign Str Programmer Indices

we can access the Elements of Series either w.r,t Programmer Defined Str Indices and also with Deafult Indices (Not Recommended in Future Version)

```

In [15]: import numpy as np
arr = np.arange(5)
print("Content of arr :- ",arr)
p_arr = pd.Series(arr ,[1,2,3,4,5]) # PROGRAMMER DEFINE INDICES (NUMBER)
print("Series :-\n",p_arr,type(p_arr))

```

Content of arr :- [0 1 2 3 4]

Series :-

```

1      0
2      1
3      2
4      3
5      4

```

dtype: int32 <class 'pandas.core.series.Series'>

```

In [16]: import numpy as np
arr = np.arange(5)

```

```
print("Content of arr :- ",arr)
p_arr = pd.Series(arr ,["A","B","C","D","E"]) # PROGRAMMER DEFINE INDICES (STR)
print("Series :-\n",p_arr,type(p_arr))
```

Content of arr :- [0 1 2 3 4]

Series :-

```
A    0
B    1
C    2
D    3
E    4
```

dtype: int32 <class 'pandas.core.series.Series'>

```
In [17]: print(p_arr["B"])
```

1

Attribute in Series

- 1.name
- 2.ndim
- 3.shape
- 4.size
- 5.values
- 6.index
- 7.empty

```
In [18]: d1 = {"ID1":23,"ID2":34,"ID3":56,"ID4":78}
var = pd.Series(d1,name ="psr")
print(var,type(var))
```

```
ID1    23
ID2    34
ID3    56
ID4    78
```

Name: psr, dtype: int64 <class 'pandas.core.series.Series'>

```
In [19]: var.name # it returns the name of Series object
```

Out[19]: 'psr'

```
In [20]: var.ndim # it returns the dimension of given series
```

Out[20]: 1

```
In [21]: var.size # it return number of element present in given series
```

Out[21]: 4

```
In [22]: var.shape # it returns the shape of given series
```

Out[22]: (4,)

```
In [23]: var.index # it returens the list of all indices of series object elements
```

```
Out[23]: Index(['ID1', 'ID2', 'ID3', 'ID4'], dtype='object')
```

```
In [24]: var.empty # it returns True if series is empty and False if series is not empty
```

```
Out[24]: False
```

Get and Set Series values

```
In [25]: # Accessing series elements using positional indexing  
s = pd.Series(np.arange(20,100,5))  
print(s)
```

```
0    20  
1    25  
2    30  
3    35  
4    40  
5    45  
6    50  
7    55  
8    60  
9    65  
10   70  
11   75  
12   80  
13   85  
14   90  
15   95  
dtype: int32
```

```
In [26]: s[0]
```

```
Out[26]: 20
```

```
In [27]: print(s[8])
```

```
60
```

```
In [28]: # accessing last value  
print(s[15])
```

```
95
```

```
In [29]: # accessing a group of value  
print(s[8::])
```

```
8    60  
9    65  
10   70  
11   75  
12   80  
13   85  
14   90  
15   95  
dtype: int32
```

```
In [30]: print(s[15::-1]) # reversing the series
```

```

15    95
14    90
13    85
12    80
11    75
10    70
9     65
8     60
7     55
6     50
5     45
4     40
3     35
2     30
1     25
0     20
dtype: int32

```

```

In [31]: # Let's create a programmer define indices Series
s = pd.Series(["Pradeep", "Rohit", "Virat", "Yashashwi", "yuraj"], ["a", "b", "c", "d", "e"])
print(s)

```

```

a    Pradeep
b     Rohit
c     Virat
d  Yashashwi
e     yuraj
dtype: object

```

```

In [32]: print(s['a'])

```

```
Pradeep
```

```

In [33]: print(s["b"])

```

```
Rohit
```

```

In [34]: print(s["b": :])

```

```

b     Rohit
c     Virat
d  Yashashwi
e     yuraj
dtype: object

```

```

In [35]: print(s["b": "d"])

```

```

b     Rohit
c     Virat
d  Yashashwi
dtype: object

```

```

In [36]: print(s["d": "b": -1])

```

```

d  Yashashwi
c     Virat
b     Rohit
dtype: object

```

```

In [37]: print(s[::-1])

```

```
e      yuraj
d      Yashashwi
c      Virat
b      Rohit
a      Pradeep
dtype: object
```

```
In [38]: # changing element in series
```

```
In [39]: print(s)
```

```
a      Pradeep
b      Rohit
c      Virat
d      Yashashwi
e      yuraj
dtype: object
```

```
In [40]: # update virat to mukesh
s["c"] = "Mukesh"
```

```
In [41]: print(s)
```

```
a      Pradeep
b      Rohit
c      Mukesh
d      Yashashwi
e      yuraj
dtype: object
```

```
In [42]: # update pradeep,yuraj,rohit
s[['a','e','b']] = ['ankit','shivam','ravi']
```

```
In [43]: print(s)
```

```
a      ankit
b      ravi
c      Mukesh
d      Yashashwi
e      shivam
dtype: object
```

```
In [44]: s[[0,1,2]] = ['apple','boy','civil'] # FutureWarning:
```

C:\Users\Pradeep Rawat\AppData\Local\Temp\ipykernel_2268\615367837.py:1: FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To set a value by position, use `ser.iloc[pos] = value`

```
s[[0,1,2]] = ['apple','boy','civil'] # FutureWarning:
```

```
In [45]: s
```

```
Out[45]: a      apple
b      boy
c      civil
d      Yashashwi
e      shivam
dtype: object
```

special extra functions

```
In [46]: # statistical functions
```

```
In [47]: s = pd.Series(np.random.rand(30))
```

```
In [48]: print("Content of series\n",s)
print("*****50)
print("maximum value in series :- ",s.max())
print("Minimum value in series :- ",s.min())
print("Avg value in series :- ",s.mean())
print("median value in series :- ",s.median())
print("Standard deviation of series :-",s.std())
print("Variance of series :- ",s.var())
print("Product of series :- ",s.product())
print("*****50)
```

Content of series

```
0    0.953410
1    0.311984
2    0.171884
3    0.093114
4    0.465691
5    0.242809
6    0.279499
7    0.592126
8    0.797437
9    0.219364
10   0.196054
11   0.216296
12   0.573966
13   0.867996
14   0.307077
15   0.637000
16   0.939323
17   0.629282
18   0.090149
19   0.567668
20   0.188537
21   0.088955
22   0.099923
23   0.950954
24   0.615849
25   0.835067
26   0.642146
27   0.732380
28   0.640930
29   0.874597
```

dtype: float64

```
*****
maximum value in series :- 0.9534095335317903
Minimum value in series :- 0.08895534179701436
Avg value in series :- 0.4940489495760368
median value in series :- 0.5708170604206741
Standard deviation of series :- 0.29631282330366837
Variance of series :- 0.08780128925419098
Product of series :- 4.209127655041258e-13
*****
```



```
In [49]: # aggregate function in which we calculate all statistical function in a single me
s.agg(['max','min','product','std','var'])
```

```
Out[49]: max          9.534095e-01
min          8.895534e-02
product      4.209128e-13
std          2.963128e-01
var          8.780129e-02
dtype: float64
```

```
In [50]: # create a new Series
s= pd.Series(np.arange(20,35,2))
print(s)
```

```
0    20
1    22
2    24
3    26
4    28
5    30
6    32
7    34
dtype: int32
```

```
In [51]: # sum of all elements
print("Sum of all elements :-",s.sum())
```

```
Sum of all elements :- 216
```

```
In [52]: s[8] = np.nan
s
```

```
Out[52]: 0    20.0
1    22.0
2    24.0
3    26.0
4    28.0
5    30.0
6    32.0
7    34.0
8     NaN
dtype: float64
```

```
In [53]: print("Number of elements :-",s.size)
```

```
Number of elements :- 9
```

```
In [54]: print("Number of numerical value :-",s.count())
```

```
Number of numerical value :- 8
```

```
In [55]: s.isnull()
```

```
Out[55]: 0    False
         1    False
         2    False
         3    False
         4    False
         5    False
         6    False
         7    False
         8     True
         dtype: bool
```

```
In [56]: s.isna()
```

```
Out[56]: 0    False
         1    False
         2    False
         3    False
         4    False
         5    False
         6    False
         7    False
         8     True
         dtype: bool
```

```
In [57]: s.cumprod()
```

```
Out[57]: 0    2.000000e+01
         1    4.400000e+02
         2    1.056000e+04
         3    2.745600e+05
         4    7.687680e+06
         5    2.306304e+08
         6    7.380173e+09
         7    2.509259e+11
         8             NaN
         dtype: float64
```

```
In [58]: # create new series
         s = pd.Series(np.random.rand(8))
```

```
In [59]: s
```

```
Out[59]: 0    0.095353
         1    0.953574
         2    0.188854
         3    0.158882
         4    0.171104
         5    0.520358
         6    0.901962
         7    0.919721
         dtype: float64
```

```
In [60]: # sort the values in ascending order
         s.sort_values()
```

```
Out[60]: 0    0.095353
         3    0.158882
         4    0.171104
         2    0.188854
         5    0.520358
         6    0.901962
         7    0.919721
         1    0.953574
         dtype: float64
```

```
In [61]: # update the s according to sorting
s.sort_values(inplace=True)
s
```

```
Out[61]: 0    0.095353
         3    0.158882
         4    0.171104
         2    0.188854
         5    0.520358
         6    0.901962
         7    0.919721
         1    0.953574
         dtype: float64
```

```
In [62]: # sort series in descending order
s.sort_values(ascending=False)
```

```
Out[62]: 1    0.953574
         7    0.919721
         6    0.901962
         5    0.520358
         2    0.188854
         4    0.171104
         3    0.158882
         0    0.095353
         dtype: float64
```

```
In [63]: # update the series
s.sort_values(ascending=False,inplace = True)
```

```
In [64]: s
```

```
Out[64]: 1    0.953574
         7    0.919721
         6    0.901962
         5    0.520358
         2    0.188854
         4    0.171104
         3    0.158882
         0    0.095353
         dtype: float64
```

```
In [65]: # create one another series
```

```
In [66]: s = pd.Series(["Guava", "Mango", "Apple", "Banana", "Guava", "Mango", "Pineapple", "App
s.sort_values()
```

```
Out[66]: 2      Apple
        7      Apple
        3      Banana
        0      Guava
        4      Guava
        1      Mango
        5      Mango
        6      Pineapple
        8      Zoo
        dtype: object
```

```
In [67]: s.max() # accroding to first alphabet
```

```
Out[67]: 'Zoo'
```

```
In [68]: s.min()
```

```
Out[68]: 'Apple'
```

```
In [69]: # description
        s.describe()
```

```
Out[69]: count      9
        unique      6
        top      Guava
        freq      2
        dtype: object
```

```
In [70]: s.head(2)
```

```
Out[70]: 0      Guava
        1      Mango
        dtype: object
```

```
In [71]: s.tail(2)
```

```
Out[71]: 7      Apple
        8      Zoo
        dtype: object
```

```
In [72]: # create one another numeircal series
        s = pd.Series(np.arange(11,23))
        print(s)
```

```
0      11
1      12
2      13
3      14
4      15
5      16
6      17
7      18
8      19
9      20
10     21
11     22
dtype: int32
```

```
In [73]: s.describe()
```

```
Out[73]: count    12.000000  
         mean     16.500000  
         std      3.605551  
         min      11.000000  
         25%      13.750000  
         50%      16.500000  
         75%      19.250000  
         max      22.000000  
         dtype: float64
```

```
In [74]: s.nlargest(3)
```

```
Out[74]: 11    22  
         10    21  
         9     20  
         dtype: int32
```

```
In [75]: s.head().nlargest(3)
```

```
Out[75]: 4     15  
         3     14  
         2     13  
         dtype: int32
```

```
In [76]: s.tail().nlargest()
```

```
Out[76]: 11    22  
         10    21  
         9     20  
         8     19  
         7     18  
         dtype: int32
```

```
In [77]: s.head(8)
```

```
Out[77]: 0     11  
         1     12  
         2     13  
         3     14  
         4     15  
         5     16  
         6     17  
         7     18  
         dtype: int32
```

```
In [78]: s.nlargest(8)
```

```
Out[78]: 11    22  
         10    21  
         9     20  
         8     19  
         7     18  
         6     17  
         5     16  
         4     15  
         dtype: int32
```

```
In [79]: s.nsmallest(4)
```

```
Out[79]: 0    11
         1    12
         2    13
         3    14
         dtype: int32
```

```
In [80]: s.nsmallest(20)
```

```
Out[80]: 0    11
         1    12
         2    13
         3    14
         4    15
         5    16
         6    17
         7    18
         8    19
         9    20
        10    21
        11    22
         dtype: int32
```

DataFrame

=> DataFrame is one of the class and whose object allows us to share the data in tabular format or 2-Dimensional format and where we can perform data analysis.

=> An object of DataFrame is MUTABLE bcoz we can do many more operation at same address

Syntax : dataframeobj = pandas.DataFrame(iterable_object,index,columns)

```
In [81]: # Number of approach to create a DataFrame
         #Creating an object of DataFrame by using int object
         a = 1
         print(a,type(a))
         print("-----")
         df=pd.DataFrame([a])
         print(df)
         print(type(df))
```

```
1 <class 'int'>
-----
0
0 1
<class 'pandas.core.frame.DataFrame'>
```

```
In [82]: #Creating an object of DataFrame by using float object
         a = 10.5
         print(a,type(a))
         print("-----")
         df=pd.DataFrame([a])
         print(df)
         print(type(df))
```

```
10.5 <class 'float'>
-----
0
0 10.5
<class 'pandas.core.frame.DataFrame'>
```

```
In [83]: #Creating an object of DataFrame by using bool object
a = True
print(a,type(a))
print("-----")
df=pd.DataFrame([a])
print(df)
print(type(df))
```

```
True <class 'bool'>
-----
0
0 True
<class 'pandas.core.frame.DataFrame'>
```

```
In [84]: #Creating an object of DataFrame by using complex object
a = 2+3j
print(a,type(a))
print("-----")
df=pd.DataFrame([a])
print(df)
print(type(df))
```

```
(2+3j) <class 'complex'>
-----
0
0 2.0+3.0j
<class 'pandas.core.frame.DataFrame'>
```

```
In [85]: #Creating an object of DataFrame by using str object
a= "Pradeep"
print(a,type(a))
print("-----")
df=pd.DataFrame([a])
print(df)
print(type(df))
```

```
Pradeep <class 'str'>
-----
0
0 Pradeep
<class 'pandas.core.frame.DataFrame'>
```

```
In [86]: #Creating an object of DataFrame by using range object
a= range(10,20,2)
print(a,type(a))
print("-----")
df=pd.DataFrame(a)
print(df)
print(type(df))
```

```
range(10, 20, 2) <class 'range'>
-----
0
0 10
1 12
2 14
3 16
4 18
<class 'pandas.core.frame.DataFrame'>
```

```
In [87]: #Creating an object of DataFrame by using List object
a = [10,20,30,40,50]
print(a,type(a))
print("-----")
df=pd.DataFrame(a)
print(df)
print(type(df))
```

```
[10, 20, 30, 40, 50] <class 'list'>
-----
0
0 10
1 20
2 30
3 40
4 50
<class 'pandas.core.frame.DataFrame'>
```

```
In [88]: #Creating an object of DataFrame by using nested List object
a = [[10,20,30,40,50],["Pradeep","Shashikant","Google","Amazon","python"]]
print(a,type(a))
print("-----")
df=pd.DataFrame(a)
print(df)
print(type(df))
```

```
[[10, 20, 30, 40, 50], ['Pradeep', 'Shashikant', 'Google', 'Amazon', 'python']] <
class 'list'>
-----
      0      1      2      3      4
0    10    20    30    40    50
1  Pradeep Shashikant Google Amazon python
<class 'pandas.core.frame.DataFrame'>
```

```
In [89]: #Creating an object of DataFrame by using nested List object and programmer defi
a = [[10,20,30,40,50],["Pradeep","Shashikant","Google","Amazon","python"]]
print(a,type(a))
print("-----")
df=pd.DataFrame(a,index = ["id","Name"])
print(df)
print(type(df))
```

```
[[10, 20, 30, 40, 50], ['Pradeep', 'Shashikant', 'Google', 'Amazon', 'python']] <
class 'list'>
-----
      id      0      1      2      3      4
id    10    20    30    40    50
Name  Pradeep Shashikant Google Amazon python
<class 'pandas.core.frame.DataFrame'>
```



```
In [90]: #Creating an object of DataFrame by using nested List object and programmer defi
a= [[10,20,30,40,50],["Pradeep","Shashikant","Google","Amazon","python"]]
print(a,type(a))
print("-----")
df=pd.DataFrame(a,index = ["id","Name"],columns = [ val for val in "ABCDE"])
print(df)
print(type(df))
```

```
[[10, 20, 30, 40, 50], ['Pradeep', 'Shashikant', 'Google', 'Amazon', 'python']] <
class 'list'>
```

```
-----
      A      B      C      D      E
id    10    20    30    40    50
Name Pradeep Shashikant Google Amazon python
<class 'pandas.core.frame.DataFrame'>
```

```
In [91]: #Creating an object of DataFrame by using List object
lst=[[100,"RS"],[200,"TR"],[300,"JH"],[400,"DR"]]
df=pd.DataFrame(lst)
print(df)
```

```
      0      1
0  100  RS
1  200  TR
2  300  JH
3  400  DR
```

```
In [92]: #Creating an object of DataFrame by using List object
lst=[[100,"RS"],[200,"TR"],[300,"JH"],[400,"DR"]]
df=pd.DataFrame(lst,columns=["id","name"])
print(df)
```

```
      id name
0  100  RS
1  200  TR
2  300  JH
3  400  DR
```

```
In [93]: #Create DataFrame for Banks VS Loans
BL=[[8.9,7.8,9.0],[7.8,7.2,9.5],[7.7,7.9,10.4]]
df=pd.DataFrame(BL,index=["SBI","ICICI","HDFC"],columns=["CARLOAN","HOMELOAN","P
print(df)
```

```
      CARLOAN  HOMELOAN  PERSONALLOAN
SBI         8.9         7.8           9.0
ICICI        7.8         7.2           9.5
HDFC        7.7         7.9          10.4
```

```
In [94]: # create an object of DataFrame by using tuple object
student_info = ("Pradeep",100, 34)
df = pd.DataFrame(student_info)
print(df)
```

```
      0
0  Pradeep
1     100
2     34
```

```
In [95]: # create an object of DataFrame by using set object
a= {10,20,302,203,32,20,10}
print(a,type(a))
```

```
df= pd.DataFrame(a)
print(df)
```

```
{32, 20, 10, 203, 302} <class 'set'>
0
0 32
1 20
2 10
3 203
4 302
```

```
In [96]: # create an object of DataFrame by using Dict object
a= {"S_Name":["Pradeep Rawat","Shashikant Sahu","Naman Rathore","Devendra Lodhi"]
print(a , type(a))
print("-----")
df = pd.DataFrame(a)
print(df)
```

```
{'S_Name': ['Pradeep Rawat', 'Shashikant Sahu', 'Naman Rathore', 'Devendra Lodhi'], 'S_Marks': [98, 97, 78, 87]} <class 'dict'>
```

```
-----
      S_Name  S_Marks
0  Pradeep Rawat    98
1  Shashikant Sahu   97
2   Naman Rathore   78
3  Devendra Lodhi   87
```

```
In [97]: # create an object of DataFrame by using ndarray object
a= np.random.rand(20).reshape(4,5)
print(a,type(a))
print("***100")
df= pd.DataFrame(a)
print(df)
print(type(df))
```

```
[[0.02224191 0.64823867 0.91499738 0.80612954 0.51140217]
 [0.31325788 0.94446867 0.33275335 0.41343166 0.328631 ]
 [0.34976871 0.1250619 0.83197462 0.55404237 0.70501492]
 [0.23277815 0.06380958 0.07166937 0.45025808 0.90189718]] <class 'numpy.ndarray'>
```

```
*****
*****
```

```
      0      1      2      3      4
0 0.022242 0.648239 0.914997 0.806130 0.511402
1 0.313258 0.944469 0.332753 0.413432 0.328631
2 0.349769 0.125062 0.831975 0.554042 0.705015
3 0.232778 0.063810 0.071669 0.450258 0.901897
<class 'pandas.core.frame.DataFrame'>
```

```
In [98]: # create an object of DataFrame by using Series object
a= pd.Series(np.arange(20,100,10))
print(a,type(a))
print("***100")
df = pd.DataFrame(a)
print(df)
print(type(df))
```

```

0    20
1    30
2    40
3    50
4    60
5    70
6    80
7    90
dtype: int32 <class 'pandas.core.series.Series'>
*****
*****
0
0    20
1    30
2    40
3    50
4    60
5    70
6    80
7    90
<class 'pandas.core.frame.DataFrame'>

```

Attributes of DataFrame

These are the attributes of the dataframe:

```

index
columns
axes
dtypes
size
shape
ndim
empty
T
values

```

```

In [99]: # Let create a DataFrame
a=[
    [112,234,456,789,980],[109,200,350,567,890],[200,234,567,234,133]
]
df = pd.DataFrame(a , index = ["CAR LOAN","HOME LOAN","GOLD LOAN"], columns = ["
df

```

```

Out[99]:

```

	SBI	HDFC	IDBI	BOB	PNB
CAR LOAN	112	234	456	789	980
HOME LOAN	109	200	350	567	890
GOLD LOAN	200	234	567	234	133

```

In [100... df.index # this returns indices values

```

```

Out[100... Index(['CAR LOAN', 'HOME LOAN', 'GOLD LOAN'], dtype='object')

```

In [101... `df.columns` # *this returns columns values*

Out[101... Index(['SBI', 'HDFC', 'IDBI', 'BOB', 'PNB'], dtype='object')

In [102... `df.axes` # *this returns index and columns*

Out[102... [Index(['CAR LOAN', 'HOME LOAN', 'GOLD LOAN'], dtype='object'),
Index(['SBI', 'HDFC', 'IDBI', 'BOB', 'PNB'], dtype='object')]

In [103... `df.dtypes` # *this returns datatypes of all columns*

Out[103... SBI int64
HDFC int64
IDBI int64
BOB int64
PNB int64
dtype: object

In [104... `df.size` # *it return column * index or total value of dataset*

Out[104... 15

In [105... `df.shape` # *it return index, columns*

Out[105... (3, 5)

In [106... `df.ndim` # *it return dimension of dataframe*

Out[106... 2

In [107... `df.empty` # *it return True if dataframe is empty and False if not empty*

Out[107... False

In [108... `df.T` # *it transpose dataframe*

Out[108...

	CAR LOAN	HOME LOAN	GOLD LOAN
SBI	112	109	200
HDFC	234	200	234
IDBI	456	350	567
BOB	789	567	234
PNB	980	890	133

In [109... `df.values` # *it return dataset of dataframe*

Out[109... array([[112, 234, 456, 789, 980],
[109, 200, 350, 567, 890],
[200, 234, 567, 234, 133]], dtype=int64)

Accessing the data from DataFrame

```
In [111... #Read the Data from CSV File---studentmarks1.csv
df = pd.read_csv("studentmarks1.csv")
df
```

```
Out[111...
   htno  name  telugu  english  hindi  maths  science  social
0    100  Ramesh     50      60     66     98      66     55
1    101  Rajesh     45      67     34     67      66     78
2    102  Rossum     56      88     56     99      44     77
3    103    Raji     56      78     34     56      88     55
4    104  Kalyan     51      63     62     93      67     51
5    105  Karthik     48      62     39     68      65     88
6    106  Kambli     53      81     59     92      48     73
7    107  Praveen     46      88     74     86      78     45
8    108  Ganesh     53      62     76     88      76     35
9    109    Nags     55      77     44     77      86     58
10   110  Biswa     66      48     86     95      48     47
11   111  Ritchi     66      68     64     76      98     75
12   104  Kalyan     51      63     62     93      67     51
13   112  shareef     50      63     99     90      76     67
14   113    sonu     60      89     98     87      77     68
15   114  Rajesh     45      67     77     55      66     46
16   115  Rakesh     67      78     88     78      67     49
```

```
In [112... #Get First 5 Records of DataFrame by using head()
df.head()
```

```
Out[112...
   htno  name  telugu  english  hindi  maths  science  social
0    100  Ramesh     50      60     66     98      66     55
1    101  Rajesh     45      67     34     67      66     78
2    102  Rossum     56      88     56     99      44     77
3    103    Raji     56      78     34     56      88     55
4    104  Kalyan     51      63     62     93      67     51
```

```
In [113... #Get First 8 Records of DataFrame by using head()
df.head(8)
```

Out[113...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45

In [114...

```
#Get Last 5 Records of DataFrame by using tail()
df.tail()
```

Out[114...

	htno	name	telugu	english	hindi	maths	science	social
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [115...

```
#Get Last 3 Records of DataFrame by using tail()
df.tail(3)
```

Out[115...

	htno	name	telugu	english	hindi	maths	science	social
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [116...

df

Out[116...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [117...

```
#Iterate the Rows the from DataFarme
for record in df.iterrows():
    print("-----")
    print(record)
```

```
-----
(0, htno      100
name      Ramesh
telugu      50
english     60
hindi       66
maths       98
science     66
social      55
Name: 0, dtype: object)
-----
(1, htno      101
name      Rajesh
telugu     45
english    67
hindi      34
maths      67
science    66
social     78
Name: 1, dtype: object)
-----
(2, htno      102
name      Rossum
telugu     56
english    88
hindi      56
maths      99
science    44
social     77
Name: 2, dtype: object)
-----
(3, htno      103
name      Raji
telugu     56
english    78
hindi      34
maths      56
science    88
social     55
Name: 3, dtype: object)
-----
(4, htno      104
name      Kalyan
telugu     51
english    63
hindi      62
maths      93
science    67
social     51
Name: 4, dtype: object)
-----
(5, htno      105
name      Karthik
telugu     48
english    62
hindi      39
maths      68
science    65
social     88
Name: 5, dtype: object)
```



```

-----
(6, htno      106
name      Kambli
telugu      53
english      81
hindi      59
maths      92
science      48
social      73
Name: 6, dtype: object)
-----
(7, htno      107
name      Praveen
telugu      46
english      88
hindi      74
maths      86
science      78
social      45
Name: 7, dtype: object)
-----
(8, htno      108
name      Ganesh
telugu      53
english      62
hindi      76
maths      88
science      76
social      35
Name: 8, dtype: object)
-----
(9, htno      109
name      Nags
telugu      55
english      77
hindi      44
maths      77
science      86
social      58
Name: 9, dtype: object)
-----
(10, htno      110
name      Biswa
telugu      66
english      48
hindi      86
maths      95
science      48
social      47
Name: 10, dtype: object)
-----
(11, htno      111
name      Ritchi
telugu      66
english      68
hindi      64
maths      76
science      98
social      75
Name: 11, dtype: object)

```

```

-----
(12, htno      104
name      Kalyan
telugu      51
english     63
hindi       62
maths       93
science     67
social      51
Name: 12, dtype: object)
-----
(13, htno      112
name      shareef
telugu     50
english    63
hindi      99
maths      90
science    76
social     67
Name: 13, dtype: object)
-----
(14, htno      113
name      sonu
telugu     60
english    89
hindi      98
maths      87
science    77
social     68
Name: 14, dtype: object)
-----
(15, htno      114
name      Rajesh
telugu     45
english    67
hindi      77
maths      55
science    66
social     46
Name: 15, dtype: object)
-----
(16, htno      115
name      Rakesh
telugu     67
english    78
hindi      88
maths      78
science    67
social     49
Name: 16, dtype: object)

```

In [118... `df.shape`

Out[118... `(17, 8)`

In [119... `df.describe()`

Out[119...

	htno	telugu	english	hindi	maths	science	social
count	17.000000	17.000000	17.000000	17.000000	17.000000	17.000000	17.000000
mean	107.294118	54.000000	70.705882	65.764706	82.235294	69.588235	59.882353
std	4.687279	7.141428	11.671358	20.504662	13.917298	14.339887	14.696438
min	100.000000	45.000000	48.000000	34.000000	55.000000	44.000000	35.000000
25%	104.000000	50.000000	63.000000	56.000000	76.000000	66.000000	49.000000
50%	107.000000	53.000000	67.000000	64.000000	87.000000	67.000000	55.000000
75%	111.000000	56.000000	78.000000	77.000000	93.000000	77.000000	73.000000
max	115.000000	67.000000	89.000000	99.000000	99.000000	98.000000	88.000000

In [120...

df

Out[120...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

Apply Indexing on DataFrame---

DataFrameObject[StartIndex:EndIndex:Step]

In [121...

df[::2]

Out[121...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
2	102	Rossum	56	88	56	99	44	77
4	104	Kalyan	51	63	62	93	67	51
6	106	Kambli	53	81	59	92	48	73
8	108	Ganesh	53	62	76	88	76	35
10	110	Biswa	66	48	86	95	48	47
12	104	Kalyan	51	63	62	93	67	51
14	113	sonu	60	89	98	87	77	68
16	115	Rakesh	67	78	88	78	67	49

In [122...

df[1::2]

Out[122...

	htno	name	telugu	english	hindi	maths	science	social
1	101	Rajesh	45	67	34	67	66	78
3	103	Raji	56	78	34	56	88	55
5	105	Karthik	48	62	39	68	65	88
7	107	Praveen	46	88	74	86	78	45
9	109	Nags	55	77	44	77	86	58
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
15	114	Rajesh	45	67	77	55	66	46

In [123...

```
# get the records from 8 to 11
df[8:12]
```

Out[123...

	htno	name	telugu	english	hindi	maths	science	social
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75

In [124...

```
# get the records of 5
df[5:6]
```

Out[124...

	htno	name	telugu	english	hindi	maths	science	social
5	105	Karthik	48	62	39	68	65	88

In [125...

```
# reverse the dataframe
df[::-1]
```

Out[125...

	htno	name	telugu	english	hindi	maths	science	social
16	115	Rakesh	67	78	88	78	67	49
15	114	Rajesh	45	67	77	55	66	46
14	113	sonu	60	89	98	87	77	68
13	112	shareef	50	63	99	90	76	67
12	104	Kalyan	51	63	62	93	67	51
11	111	Ritchi	66	68	64	76	98	75
10	110	Biswa	66	48	86	95	48	47
9	109	Nags	55	77	44	77	86	58
8	108	Ganesh	53	62	76	88	76	35
7	107	Praveen	46	88	74	86	78	45
6	106	Kambli	53	81	59	92	48	73
5	105	Karthik	48	62	39	68	65	88
4	104	Kalyan	51	63	62	93	67	51
3	103	Raji	56	78	34	56	88	55
2	102	Rossum	56	88	56	99	44	77
1	101	Rajesh	45	67	34	67	66	78
0	100	Ramesh	50	60	66	98	66	55

In [126...

df

Out[126...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [127...

```
# get the all names
df["name"]
```

Out[127...

```
0    Ramesh
1    Rajesh
2    Rossum
3     Raji
4    Kalyan
5    Karthik
6    Kambli
7    Praveen
8    Ganesh
9     Nags
10    Biswa
11    Ritchi
12    Kalyan
13  shareef
14     sonu
15    Rajesh
16    Rakesh
Name: name, dtype: object
```

In [128...

```
# get name and math marks
df[['maths', 'name']]
```

Out[128...

	maths	name
0	98	Ramesh
1	67	Rajesh
2	99	Rossum
3	56	Raji
4	93	Kalyan
5	68	Karthik
6	92	Kambli
7	86	Praveen
8	88	Ganesh
9	77	Nags
10	95	Biswa
11	76	Ritchi
12	93	Kalyan
13	90	shareef
14	87	sonu
15	55	Rajesh
16	78	Rakesh

In [129...

```
# get even number indexes english , math , htno
df[::2][["htno", "name", "maths", "english"]]
```

Out[129...

	htno	name	maths	english
0	100	Ramesh	98	60
2	102	Rossum	99	88
4	104	Kalyan	93	63
6	106	Kambli	92	81
8	108	Ganesh	88	62
10	110	Biswa	95	48
12	104	Kalyan	93	63
14	113	sonu	87	89
16	115	Rakesh	78	78

In [130...

```
# get odd number indexes english , math , htno
df[["htno", "name", "maths", "english"]][1::2]
```

Out[130...

	htno	name	maths	english
1	101	Rajesh	67	67
3	103	Raji	56	78
5	105	Karthik	68	62
7	107	Praveen	86	88
9	109	Nags	77	77
11	111	Ritchi	76	68
13	112	shareef	90	63
15	114	Rajesh	55	67

In [131...

df[10:2]

Out[131...

	htno	name	telugu	english	hindi	maths	science	social
--	------	------	--------	---------	-------	-------	---------	--------

In [132...

df[10:2:-1]

Out[132...

	htno	name	telugu	english	hindi	maths	science	social
10	110	Biswa	66	48	86	95	48	47
9	109	Nags	55	77	44	77	86	58
8	108	Ganesh	53	62	76	88	76	35
7	107	Praveen	46	88	74	86	78	45
6	106	Kambli	53	81	59	92	48	73
5	105	Karthik	48	62	39	68	65	88
4	104	Kalyan	51	63	62	93	67	51
3	103	Raji	56	78	34	56	88	55

Understanding .loc[] -----> here start and stop index(row indices) included and colnames can be used to get the data from dataframe (but not column indices used)

In [133...

df

Out[133...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [134...

```
# get 5th records
df.loc[5]
```

Out[134...

```
htno      105
name      Karthik
telugu     48
english    62
hindi      39
maths      68
science    65
social     88
Name: 5, dtype: object
```

In [135...

```
# get 11th records
df.loc[11]
```

Out[135...

```
htno      111
name      Ritchi
telugu     66
english    68
hindi      64
maths      76
science    98
social     75
Name: 11, dtype: object
```

In [136... *# get english marks of 3 record*
`df.loc[3,["english"]]`

Out[136... english 78
 Name: 3, dtype: object

In [137... *# get science marks of 8 record*
`df.loc[8,['science']]`

Out[137... science 76
 Name: 8, dtype: object

In [138... *# get name of 10 to 15 record*
`df.loc[10:15,['name']]`

Out[138... **name**

10	Biswa
11	Ritchi
12	Kalyan
13	shareef
14	sonu
15	Rajesh

In [139... *# get data from 5 to 15*
`df.loc[5:15]`

Out[139...

	htno	name	telugu	english	hindi	maths	science	social
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46

In [140... *# get htno , name , maths , from 10 to 16*
`df.loc[10:16,['htno','name','maths']]`

Out[140...

	htno	name	maths
10	110	Biswa	95
11	111	Ritchi	76
12	104	Kalyan	93
13	112	shareef	90
14	113	sonu	87
15	114	Rajesh	55
16	115	Rakesh	78

In [141...

```
# get alternative columns data
df.loc[:, 'htno'::2]
```

Out[141...

	htno	telugu	hindi	science
0	100	50	66	66
1	101	45	34	66
2	102	56	56	44
3	103	56	34	88
4	104	51	62	67
5	105	48	39	65
6	106	53	59	48
7	107	46	74	78
8	108	53	76	76
9	109	55	44	86
10	110	66	86	48
11	111	66	64	98
12	104	51	62	67
13	112	50	99	76
14	113	60	98	77
15	114	45	77	66
16	115	67	88	67

In [142...

```
# get even columns and odd rows
df.loc[1::2, ::2]
```

Out[142...

	htno	telugu	hindi	science
1	101	45	34	66
3	103	56	34	88
5	105	48	39	65
7	107	46	74	78
9	109	55	44	86
11	111	66	64	98
13	112	50	99	76
15	114	45	77	66

In [143...

```
# get data name to science
df.loc[:, 'name' : 'science']
```

Out[143...

	name	telugu	english	hindi	maths	science
0	Ramesh	50	60	66	98	66
1	Rajesh	45	67	34	67	66
2	Rossum	56	88	56	99	44
3	Raji	56	78	34	56	88
4	Kalyan	51	63	62	93	67
5	Karthik	48	62	39	68	65
6	Kambli	53	81	59	92	48
7	Praveen	46	88	74	86	78
8	Ganesh	53	62	76	88	76
9	Nags	55	77	44	77	86
10	Biswa	66	48	86	95	48
11	Ritchi	66	68	64	76	98
12	Kalyan	51	63	62	93	67
13	shareef	50	63	99	90	76
14	sonu	60	89	98	87	77
15	Rajesh	45	67	77	55	66
16	Rakesh	67	78	88	78	67

understanding iloc[]----->

Here start index included and stop
index(row indices) excluded and col numbers(column indices)

must be used to get the data from dataframe (but not column names used)

```
In [144... #Get the 7th Record by using iloc[]
df.iloc[7]
```

```
Out[144... htno      107
name      Praveen
telugu     46
english    88
hindi      74
maths      86
science    78
social     45
Name: 7, dtype: object
```

```
In [145... # get 3rd record
df.iloc[3]
```

```
Out[145... htno      103
name      Raji
telugu     56
english    78
hindi      34
maths      56
science    88
social     55
Name: 3, dtype: object
```

```
In [146... # get 4th record name
df.iloc[4,[1]]
```

```
Out[146... name      Kalyan
Name: 4, dtype: object
```

```
In [147... # get record 3 to 9
df.iloc[3:10]
```

```
Out[147...      htno  name  telugu  english  hindi  maths  science  social
3  103    Raji    56      78     34     56      88      55
4  104   Kalyan    51      63     62     93      67      51
5  105   Karthik    48      62     39     68      65      88
6  106   Kambli    53      81     59     92      48      73
7  107   Praveen    46      88     74     86      78      45
8  108   Ganesh    53      62     76     88      76      35
9  109    Nags     55      77     44     77      86      58
```

```
In [148... # get alternative records
df.iloc[:,2]
```

Out[148...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
2	102	Rossum	56	88	56	99	44	77
4	104	Kalyan	51	63	62	93	67	51
6	106	Kambli	53	81	59	92	48	73
8	108	Ganesh	53	62	76	88	76	35
10	110	Biswa	66	48	86	95	48	47
12	104	Kalyan	51	63	62	93	67	51
14	113	sonu	60	89	98	87	77	68
16	115	Rakesh	67	78	88	78	67	49

In [149...

```
# get alternative records from 1
df.iloc[1::2]
```

Out[149...

	htno	name	telugu	english	hindi	maths	science	social
1	101	Rajesh	45	67	34	67	66	78
3	103	Raji	56	78	34	56	88	55
5	105	Karthik	48	62	39	68	65	88
7	107	Praveen	46	88	74	86	78	45
9	109	Nags	55	77	44	77	86	58
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
15	114	Rajesh	45	67	77	55	66	46

In [150...

```
# get alternaive records htno only
df.iloc[:,2,[0]]
```

Out[150...

htno	
0	100
2	102
4	104
6	106
8	108
10	110
12	104
14	113
16	115

In [151...

```
# print dataframe
df
```

Out[151...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [152...

```
# get 3 to 11 records , name, math marks, english marks
df.iloc[3:12,[1,5,3]]
```

Out[152...

	name	maths	english
3	Raji	56	78
4	Kalyan	93	63
5	Karthik	68	62
6	Kambli	92	81
7	Praveen	86	88
8	Ganesh	88	62
9	Nags	77	77
10	Biswa	95	48
11	Ritchi	76	68

In [153...

```
# get all data record name to science
df.iloc[:,1:7]
```

Out[153...

	name	telugu	english	hindi	maths	science
0	Ramesh	50	60	66	98	66
1	Rajesh	45	67	34	67	66
2	Rossum	56	88	56	99	44
3	Raji	56	78	34	56	88
4	Kalyan	51	63	62	93	67
5	Karthik	48	62	39	68	65
6	Kambli	53	81	59	92	48
7	Praveen	46	88	74	86	78
8	Ganesh	53	62	76	88	76
9	Nags	55	77	44	77	86
10	Biswa	66	48	86	95	48
11	Ritchi	66	68	64	76	98
12	Kalyan	51	63	62	93	67
13	shareef	50	63	99	90	76
14	sonu	60	89	98	87	77
15	Rajesh	45	67	77	55	66
16	Rakesh	67	78	88	78	67

In [154...

```
# get alternative column and row data
df.iloc[:,::2,::2]
```


Out[154...

	htno	telugu	hindi	science
0	100	50	66	66
2	102	56	56	44
4	104	51	62	67
6	106	53	59	48
8	108	53	76	76
10	110	66	86	48
12	104	51	62	67
14	113	60	98	77
16	115	67	88	67

In [155...

```
# get data in reverse order
df.iloc[::-1,::-1]
```

Out[155...

	social	science	maths	hindi	english	telugu	name	htno
16	49	67	78	88	78	67	Rakesh	115
15	46	66	55	77	67	45	Rajesh	114
14	68	77	87	98	89	60	sonu	113
13	67	76	90	99	63	50	shareef	112
12	51	67	93	62	63	51	Kalyan	104
11	75	98	76	64	68	66	Ritchi	111
10	47	48	95	86	48	66	Biswa	110
9	58	86	77	44	77	55	Nags	109
8	35	76	88	76	62	53	Ganesh	108
7	45	78	86	74	88	46	Praveen	107
6	73	48	92	59	81	53	Kambli	106
5	88	65	68	39	62	48	Karthik	105
4	51	67	93	62	63	51	Kalyan	104
3	55	88	56	34	78	56	Raji	103
2	77	44	99	56	88	56	Rossum	102
1	78	66	67	34	67	45	Rajesh	101
0	55	66	98	66	60	50	Ramesh	100

Adding new column to DataFrame

Syntax :- dataframeobj["new col name"] =
 default value
 dataframeobj["new col name "] =
 expression

In [156...

df

Out[156...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [157...

```
# adding new column total in the dataframe
df["Total"] = df["telugu"]+df["english"]+df["hindi"]+df["maths"]+df["science"]+d
```

In [158...

df

Out[158...

	htno	name	telugu	english	hindi	maths	science	social	Total
0	100	Ramesh	50	60	66	98	66	55	395
1	101	Rajesh	45	67	34	67	66	78	357
2	102	Rossum	56	88	56	99	44	77	420
3	103	Raji	56	78	34	56	88	55	367
4	104	Kalyan	51	63	62	93	67	51	387
5	105	Karthik	48	62	39	68	65	88	370
6	106	Kambli	53	81	59	92	48	73	406
7	107	Praveen	46	88	74	86	78	45	417
8	108	Ganesh	53	62	76	88	76	35	390
9	109	Nags	55	77	44	77	86	58	397
10	110	Biswa	66	48	86	95	48	47	390
11	111	Ritchi	66	68	64	76	98	75	447
12	104	Kalyan	51	63	62	93	67	51	387
13	112	shareef	50	63	99	90	76	67	445
14	113	sonu	60	89	98	87	77	68	479
15	114	Rajesh	45	67	77	55	66	46	356
16	115	Rakesh	67	78	88	78	67	49	427

In [159...

```
# adding new column percentage in the DataFrame
df["percentage"] = round(df['Total']/6,2)
```

In [160...

df

Out[160...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage
0	100	Ramesh	50	60	66	98	66	55	395	65.83
1	101	Rajesh	45	67	34	67	66	78	357	59.50
2	102	Rossum	56	88	56	99	44	77	420	70.00
3	103	Raji	56	78	34	56	88	55	367	61.17
4	104	Kalyan	51	63	62	93	67	51	387	64.50
5	105	Karthik	48	62	39	68	65	88	370	61.67
6	106	Kambli	53	81	59	92	48	73	406	67.67
7	107	Praveen	46	88	74	86	78	45	417	69.50
8	108	Ganesh	53	62	76	88	76	35	390	65.00
9	109	Nags	55	77	44	77	86	58	397	66.17
10	110	Biswa	66	48	86	95	48	47	390	65.00
11	111	Ritchi	66	68	64	76	98	75	447	74.50
12	104	Kalyan	51	63	62	93	67	51	387	64.50
13	112	shareef	50	63	99	90	76	67	445	74.17
14	113	sonu	60	89	98	87	77	68	479	79.83
15	114	Rajesh	45	67	77	55	66	46	356	59.33
16	115	Rakesh	67	78	88	78	67	49	427	71.17

In [161...

```
# adding new column grade with default nan
df['Grade']=np.nan
```

In [162...

```
df
```

Out[162...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
0	100	Ramesh	50	60	66	98	66	55	395	65.83	
1	101	Rajesh	45	67	34	67	66	78	357	59.50	
2	102	Rossum	56	88	56	99	44	77	420	70.00	
3	103	Raji	56	78	34	56	88	55	367	61.17	
4	104	Kalyan	51	63	62	93	67	51	387	64.50	
5	105	Karthik	48	62	39	68	65	88	370	61.67	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
8	108	Ganesh	53	62	76	88	76	35	390	65.00	
9	109	Nags	55	77	44	77	86	58	397	66.17	
10	110	Biswa	66	48	86	95	48	47	390	65.00	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
12	104	Kalyan	51	63	62	93	67	51	387	64.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
14	113	sonu	60	89	98	87	77	68	479	79.83	
15	114	Rajesh	45	67	77	55	66	46	356	59.33	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	

Data Filtering and Conditional Change

In [163...

```
# filter data whose percentage greater than 75
df[df['percentage']>75]
```

Out[163...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	Gra
14	113	sonu	60	89	98	87	77	68	479	79.83	N

In [164...

```
# filter data whose percentage between 60 and 75
df.loc[(df['percentage']< 75) & (df['percentage'] >=60)]
```

Out[164...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
0	100	Ramesh	50	60	66	98	66	55	395	65.83	
2	102	Rossum	56	88	56	99	44	77	420	70.00	
3	103	Raji	56	78	34	56	88	55	367	61.17	
4	104	Kalyan	51	63	62	93	67	51	387	64.50	
5	105	Karthik	48	62	39	68	65	88	370	61.67	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
8	108	Ganesh	53	62	76	88	76	35	390	65.00	
9	109	Nags	55	77	44	77	86	58	397	66.17	
10	110	Biswa	66	48	86	95	48	47	390	65.00	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
12	104	Kalyan	51	63	62	93	67	51	387	64.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	



In [165...

```
# filtering data whose percentage Less than 60
df.loc[df["percentage"]<60]
```

Out[165...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	Grade
1	101	Rajesh	45	67	34	67	66	78	357	59.50	N
15	114	Rajesh	45	67	77	55	66	46	356	59.33	N



In [166...

```
# now assigning grades with some rules
# Grade = A whose percentage greater than 75
# grade = B whose percentage between 60 and 75
# grade = c whose percentage Less than 60
df["Grade"]="0"
df.loc[df['percentage']>=75,['Grade']]="A"
df.loc[(df['percentage']< 75) & (df['percentage'] >=60),['Grade']] = "B"
df.loc[df["percentage"]<60,['Grade']]="C"
```

In [167...

df

Out[167...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
0	100	Ramesh	50	60	66	98	66	55	395	65.83	
1	101	Rajesh	45	67	34	67	66	78	357	59.50	
2	102	Rossum	56	88	56	99	44	77	420	70.00	
3	103	Raji	56	78	34	56	88	55	367	61.17	
4	104	Kalyan	51	63	62	93	67	51	387	64.50	
5	105	Karthik	48	62	39	68	65	88	370	61.67	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
8	108	Ganesh	53	62	76	88	76	35	390	65.00	
9	109	Nags	55	77	44	77	86	58	397	66.17	
10	110	Biswa	66	48	86	95	48	47	390	65.00	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
12	104	Kalyan	51	63	62	93	67	51	387	64.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
14	113	sonu	60	89	98	87	77	68	479	79.83	
15	114	Rajesh	45	67	77	55	66	46	356	59.33	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	

In [168...

```
df[df["maths"] > 67][['name']]
```

Out[168...

	name
0	Ramesh
2	Rossum
4	Kalyan
5	Karthik
6	Kambli
7	Praveen
8	Ganesh
9	Nags
10	Biswa
11	Ritchi
12	Kalyan
13	shareef
14	sonu
16	Rakesh

In [169...

```
# student name and math number whose math number greater than 85
df.loc[df['maths']>85][['name', 'maths']]
```

Out[169...

	name	maths
0	Ramesh	98
2	Rossum	99
4	Kalyan	93
6	Kambli	92
7	Praveen	86
8	Ganesh	88
10	Biswa	95
12	Kalyan	93
13	shareef	90
14	sonu	87

In [170...

```
# students data whose Grades a or b
df[((df["Grade"] == 'A') | (df["Grade"] == "B"))]
```


Out[170...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
0	100	Ramesh	50	60	66	98	66	55	395	65.83	
2	102	Rossum	56	88	56	99	44	77	420	70.00	
3	103	Raji	56	78	34	56	88	55	367	61.17	
4	104	Kalyan	51	63	62	93	67	51	387	64.50	
5	105	Karthik	48	62	39	68	65	88	370	61.67	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
8	108	Ganesh	53	62	76	88	76	35	390	65.00	
9	109	Nags	55	77	44	77	86	58	397	66.17	
10	110	Biswa	66	48	86	95	48	47	390	65.00	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
12	104	Kalyan	51	63	62	93	67	51	387	64.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
14	113	sonu	60	89	98	87	77	68	479	79.83	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	



In [171...

```
# students who got equal marks in math and science
df[df["hindi"] == df["science"]][['name','science','hindi']]
```

Out[171...

	name	science	hindi
0	Ramesh	66	66
8	Ganesh	76	76

In [172...

```
# get students data whose maths marks greater than science marks
df[df['maths'] >= df['science']][['name','maths','science']]
```

Out[172...

	name	maths	science
0	Ramesh	98	66
1	Rajesh	67	66
2	Rossum	99	44
4	Kalyan	93	67
5	Karthik	68	65
6	Kambli	92	48
7	Praveen	86	78
8	Ganesh	88	76
10	Biswa	95	48
12	Kalyan	93	67
13	shareef	90	76
14	sonu	87	77
16	Rakesh	78	67

In [173...

```
# get student data whose total marks greater than 400
df.loc[df['Total'] >=400]
```

Out[173...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
2	102	Rossum	56	88	56	99	44	77	420	70.00	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
14	113	sonu	60	89	98	87	77	68	479	79.83	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	



exporting DataFrame data to CSV file

Syntax :- `df.to_csv("absolute_path")`

In [174...

```
df.to_csv("Student_final_data.csv")
```

In [175...

```
# exporting DataFrame data to excel sheet
df.to_excel("Student_info.xlsx")
```

In [176...

```
# exporting dataframe data to txt file
df.to_csv("Student_informstion.txt")
```

```
In [177... # export data without indexes into txt file
df.to_csv("Student_informstion2.txt", index=False, sep = "\t")
```

```
In [178... df
```

```
Out[178... 
```

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage	G
0	100	Ramesh	50	60	66	98	66	55	395	65.83	
1	101	Rajesh	45	67	34	67	66	78	357	59.50	
2	102	Rossum	56	88	56	99	44	77	420	70.00	
3	103	Raji	56	78	34	56	88	55	367	61.17	
4	104	Kalyan	51	63	62	93	67	51	387	64.50	
5	105	Karthik	48	62	39	68	65	88	370	61.67	
6	106	Kambli	53	81	59	92	48	73	406	67.67	
7	107	Praveen	46	88	74	86	78	45	417	69.50	
8	108	Ganesh	53	62	76	88	76	35	390	65.00	
9	109	Nags	55	77	44	77	86	58	397	66.17	
10	110	Biswa	66	48	86	95	48	47	390	65.00	
11	111	Ritchi	66	68	64	76	98	75	447	74.50	
12	104	Kalyan	51	63	62	93	67	51	387	64.50	
13	112	shareef	50	63	99	90	76	67	445	74.17	
14	113	sonu	60	89	98	87	77	68	479	79.83	
15	114	Rajesh	45	67	77	55	66	46	356	59.33	
16	115	Rakesh	67	78	88	78	67	49	427	71.17	

removing columns from dataframe

```
In [179... # syntax 1 :- dataframe.drop(columns="col name")
# drop grade column from data and update dataframe
df=df.drop(columns="Grade")
df
```

Out[179...

	htno	name	telugu	english	hindi	maths	science	social	Total	percentage
0	100	Ramesh	50	60	66	98	66	55	395	65.83
1	101	Rajesh	45	67	34	67	66	78	357	59.50
2	102	Rossum	56	88	56	99	44	77	420	70.00
3	103	Raji	56	78	34	56	88	55	367	61.17
4	104	Kalyan	51	63	62	93	67	51	387	64.50
5	105	Karthik	48	62	39	68	65	88	370	61.67
6	106	Kambli	53	81	59	92	48	73	406	67.67
7	107	Praveen	46	88	74	86	78	45	417	69.50
8	108	Ganesh	53	62	76	88	76	35	390	65.00
9	109	Nags	55	77	44	77	86	58	397	66.17
10	110	Biswa	66	48	86	95	48	47	390	65.00
11	111	Ritchi	66	68	64	76	98	75	447	74.50
12	104	Kalyan	51	63	62	93	67	51	387	64.50
13	112	shareef	50	63	99	90	76	67	445	74.17
14	113	sonu	60	89	98	87	77	68	479	79.83
15	114	Rajesh	45	67	77	55	66	46	356	59.33
16	115	Rakesh	67	78	88	78	67	49	427	71.17

In [180...

```
# syntax2: df.drop(columns=["col name1","col name2" ,-----])
# remove total and percentage both
df=df.drop(columns = ["Total","percentage"])
print(df)
```

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [181...

```
# remove hindi , maths
df.drop(columns =['hindi','maths'])
```

Out[181...

	htno	name	telugu	english	science	social
0	100	Ramesh	50	60	66	55
1	101	Rajesh	45	67	66	78
2	102	Rossum	56	88	44	77
3	103	Raji	56	78	88	55
4	104	Kalyan	51	63	67	51
5	105	Karthik	48	62	65	88
6	106	Kambli	53	81	48	73
7	107	Praveen	46	88	78	45
8	108	Ganesh	53	62	76	35
9	109	Nags	55	77	86	58
10	110	Biswa	66	48	48	47
11	111	Ritchi	66	68	98	75
12	104	Kalyan	51	63	67	51
13	112	shareef	50	63	76	67
14	113	sonu	60	89	77	68
15	114	Rajesh	45	67	66	46
16	115	Rakesh	67	78	67	49

In [182...

```
# syntax :- df.drop(columns=df.columns[col number])
# drop htno using co number
df.drop(columns = df.columns[0],inplace=True)
df
```

Out[182...

	name	telugu	english	hindi	maths	science	social
0	Ramesh	50	60	66	98	66	55
1	Rajesh	45	67	34	67	66	78
2	Rossum	56	88	56	99	44	77
3	Raji	56	78	34	56	88	55
4	Kalyan	51	63	62	93	67	51
5	Karthik	48	62	39	68	65	88
6	Kambli	53	81	59	92	48	73
7	Praveen	46	88	74	86	78	45
8	Ganesh	53	62	76	88	76	35
9	Nags	55	77	44	77	86	58
10	Biswa	66	48	86	95	48	47
11	Ritchi	66	68	64	76	98	75
12	Kalyan	51	63	62	93	67	51
13	shareef	50	63	99	90	76	67
14	sonu	60	89	98	87	77	68
15	Rajesh	45	67	77	55	66	46
16	Rakesh	67	78	88	78	67	49

In [183...

```
# drop maths , science using column number
df.drop(columns=df.columns[[4,5]],inplace = True)
```

In [184...

df

Out[184...

	name	telugu	english	hindi	social
0	Ramesh	50	60	66	55
1	Rajesh	45	67	34	78
2	Rossum	56	88	56	77
3	Raji	56	78	34	55
4	Kalyan	51	63	62	51
5	Karthik	48	62	39	88
6	Kambli	53	81	59	73
7	Praveen	46	88	74	45
8	Ganesh	53	62	76	35
9	Nags	55	77	44	58
10	Biswa	66	48	86	47
11	Ritchi	66	68	64	75
12	Kalyan	51	63	62	51
13	shareef	50	63	99	67
14	sonu	60	89	98	68
15	Rajesh	45	67	77	46
16	Rakesh	67	78	88	49

Removing Row from dataframe

In [185...

```
df = pd.read_csv("studentmarks1.csv")
```

In [186...

```
df
```

Out[186...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [187...

```
# syntax :- df.drop(labels = index_value ,axis=0) , here axis =0 refers to row
# drop kalyan data
df.drop(labels = 12 , axis=0,inplace= True)
```

In [188...

df

Out[188...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [189...

```
# df.drop(labels = [index_value1,index_value2 -----],axis=0)
# drop shareef , rajesh data
df.drop(labels =[14,15],axis =0,inplace=True)
```

In [190...

df

Out[190...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
16	115	Rakesh	67	78	88	78	67	49

sorting the data of dataframe

In [191...

```
df = pd.read_csv("student_final_data.csv")
```

In [192...

```
df
```

Out[192...

	Unnamed: 0	htno	name	telugu	english	hindi	maths	science	social	Total	pe
0	0	100	Ramesh	50	60	66	98	66	55	395	
1	1	101	Rajesh	45	67	34	67	66	78	357	
2	2	102	Rossum	56	88	56	99	44	77	420	
3	3	103	Raji	56	78	34	56	88	55	367	
4	4	104	Kalyan	51	63	62	93	67	51	387	
5	5	105	Karthik	48	62	39	68	65	88	370	
6	6	106	Kambli	53	81	59	92	48	73	406	
7	7	107	Praveen	46	88	74	86	78	45	417	
8	8	108	Ganesh	53	62	76	88	76	35	390	
9	9	109	Nags	55	77	44	77	86	58	397	
10	10	110	Biswa	66	48	86	95	48	47	390	
11	11	111	Ritchi	66	68	64	76	98	75	447	
12	12	104	Kalyan	51	63	62	93	67	51	387	
13	13	112	shareef	50	63	99	90	76	67	445	
14	14	113	sonu	60	89	98	87	77	68	479	
15	15	114	Rajesh	45	67	77	55	66	46	356	
16	16	115	Rakesh	67	78	88	78	67	49	427	



In [193...

```
# sort the data according to percentage
df.sort_values("percentage")
```

Out[193...

	Unnamed: 0	htno	name	telugu	english	hindi	maths	science	social	Total	pe
15	15	114	Rajesh	45	67	77	55	66	46	356	
1	1	101	Rajesh	45	67	34	67	66	78	357	
3	3	103	Raji	56	78	34	56	88	55	367	
5	5	105	Karthik	48	62	39	68	65	88	370	
12	12	104	Kalyan	51	63	62	93	67	51	387	
4	4	104	Kalyan	51	63	62	93	67	51	387	
8	8	108	Ganesh	53	62	76	88	76	35	390	
10	10	110	Biswa	66	48	86	95	48	47	390	
0	0	100	Ramesh	50	60	66	98	66	55	395	
9	9	109	Nags	55	77	44	77	86	58	397	
6	6	106	Kambli	53	81	59	92	48	73	406	
7	7	107	Praveen	46	88	74	86	78	45	417	
2	2	102	Rossum	56	88	56	99	44	77	420	
16	16	115	Rakesh	67	78	88	78	67	49	427	
13	13	112	shareef	50	63	99	90	76	67	445	
11	11	111	Ritchi	66	68	64	76	98	75	447	
14	14	113	sonu	60	89	98	87	77	68	479	



In [194...

```
df.sort_values("percentage",ascending=False,inplace=True)
```

In [195...

```
df
```

Out[195...

	Unnamed: 0	htno	name	telugu	english	hindi	maths	science	social	Total	pe
14	14	113	sonu	60	89	98	87	77	68	479	
11	11	111	Ritchi	66	68	64	76	98	75	447	
13	13	112	shareef	50	63	99	90	76	67	445	
16	16	115	Rakesh	67	78	88	78	67	49	427	
2	2	102	Rossum	56	88	56	99	44	77	420	
7	7	107	Praveen	46	88	74	86	78	45	417	
6	6	106	Kambli	53	81	59	92	48	73	406	
9	9	109	Nags	55	77	44	77	86	58	397	
0	0	100	Ramesh	50	60	66	98	66	55	395	
10	10	110	Biswa	66	48	86	95	48	47	390	
8	8	108	Ganesh	53	62	76	88	76	35	390	
12	12	104	Kalyan	51	63	62	93	67	51	387	
4	4	104	Kalyan	51	63	62	93	67	51	387	
5	5	105	Karthik	48	62	39	68	65	88	370	
3	3	103	Raji	56	78	34	56	88	55	367	
1	1	101	Rajesh	45	67	34	67	66	78	357	
15	15	114	Rajesh	45	67	77	55	66	46	356	



In [196...

```
# find top 5 toppers
df.head()
```

Out[196...

	Unnamed: 0	htno	name	telugu	english	hindi	maths	science	social	Total	pe
14	14	113	sonu	60	89	98	87	77	68	479	
11	11	111	Ritchi	66	68	64	76	98	75	447	
13	13	112	shareef	50	63	99	90	76	67	445	
16	16	115	Rakesh	67	78	88	78	67	49	427	
2	2	102	Rossum	56	88	56	99	44	77	420	



In [197...

```
# find Least markers
df.tail()[::-1]
```

Out[197...

	Unnamed: 0	htno	name	telugu	english	hindi	maths	science	social	Total	per
15	15	114	Rajesh	45	67	77	55	66	46	356	
1	1	101	Rajesh	45	67	34	67	66	78	357	
3	3	103	Raji	56	78	34	56	88	55	367	
5	5	105	Karthik	48	62	39	68	65	88	370	
4	4	104	Kalyan	51	63	62	93	67	51	387	

Finding and Removing Duplicates

In [198...

```
#Read the Data from CSV File---studentmarks1.csv
df = pd.read_csv("studentmarks1.csv")
df
```

Out[198...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
12	104	Kalyan	51	63	62	93	67	51
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [199...

```
# find duplicates
df.duplicated()
```

```
Out[199... 0    False
          1    False
          2    False
          3    False
          4    False
          5    False
          6    False
          7    False
          8    False
          9    False
         10    False
         11    False
         12     True
         13    False
         14    False
         15    False
         16    False
dtype: bool
```

```
In [200... # find duplicate data
df[df.duplicated()]
```

```
Out[200...      htno  name  telugu  english  hindi  maths  science  social
12   104  Kalyan     51        63     62     93        67     51
```

```
In [201... # remove duplicated data
df.drop_duplicates()
```

Out[201...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [202...

```
# update the dataframe
df.drop_duplicates(inplace=True)
```

In [203...

```
df
```


Out[203...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [204...

df.duplicated()

Out[204...

```

0      False
1      False
2      False
3      False
4      False
5      False
6      False
7      False
8      False
9      False
10     False
11     False
13     False
14     False
15     False
16     False
dtype: bool

```

In [205...

print(df.duplicated().sum())

0

Adding new Row to Data Frame

```
dataframeobj.loc[len(DataFrame)+1]=
[Val1,Val2....Val-n]
```

In [206...

df

Out[206...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49

In [207...

```
# add a new row in dataframe with data
df.loc[len(df)+1] = [116, "Pradeep", 70, 90, 89, 92, 67, 90]
```

In [208...

df

Out[208...

	htno	name	telugu	english	hindi	maths	science	social
0	100	Ramesh	50	60	66	98	66	55
1	101	Rajesh	45	67	34	67	66	78
2	102	Rossum	56	88	56	99	44	77
3	103	Raji	56	78	34	56	88	55
4	104	Kalyan	51	63	62	93	67	51
5	105	Karthik	48	62	39	68	65	88
6	106	Kambli	53	81	59	92	48	73
7	107	Praveen	46	88	74	86	78	45
8	108	Ganesh	53	62	76	88	76	35
9	109	Nags	55	77	44	77	86	58
10	110	Biswa	66	48	86	95	48	47
11	111	Ritchi	66	68	64	76	98	75
13	112	shareef	50	63	99	90	76	67
14	113	sonu	60	89	98	87	77	68
15	114	Rajesh	45	67	77	55	66	46
16	115	Rakesh	67	78	88	78	67	49
17	116	Pradeep	70	90	89	92	67	90

GroupBy

The Groupby mechanism in the pandas provides a way to break a dataframe into different groups or chunks based on the values of single or multiple columns

In [214...

```
df=pd.read_csv("employee.csv")
```

In [215...

```
df
```

Out[215...

	ID	Name	Age	City	Experience
0	11	Jack	44	Sydney	19
1	12	Riti	41	Delhi	17
2	13	Aadi	46	Mumbai	11
3	14	Mohit	45	Delhi	15
4	15	Veena	42	Mumbai	17
5	16	Shaunak	42	Sydney	14
6	17	manik	42	Delhi	11
7	18	vikas	42	Mumbai	15
8	19	Samir	42	Delhi	15
9	20	Shobhit	40	Sydney	12

In [216...

```
# group data on the behalf of city
df.groupby("City")
```

Out[216...

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000024E4F9C12B0>
```

In [217...

```
data = df.groupby("City")
print("Types of data :-",type(data))
```

```
Types of data :- <class 'pandas.core.groupby.generic.DataFrameGroupBy'>
```

In [218...

```
for val in data:
    print(val,type(val))
    print("-----")
```

```
('Delhi',   ID  Name  Age  City  Experience
1  12  Riti   41  Delhi      17
3  14  Mohit  45  Delhi      15
6  17  manik  42  Delhi      11
8  19  Samir  42  Delhi      15) <class 'tuple'>
-----
('Mumbai',  ID  Name  Age  City  Experience
2  13  Aadi   46  Mumbai     11
4  15  Veena  42  Mumbai     17
7  18  vikas  42  Mumbai     15) <class 'tuple'>
-----
('Sydney',  ID      Name  Age  City  Experience
0  11    Jack   44  Sydney     19
5  16  Shaunak  42  Sydney     14
9  20  Shobhit  40  Sydney     12) <class 'tuple'>
-----
```

In [219...

```
for val in data :
    print("group name :-",val[0])
    print("***50")
    print(val[1])
    print("***50")
```

```

group name :- Delhi
*****
   ID  Name  Age  City  Experience
1  12   Riti  41  Delhi         17
3  14  Mohit  45  Delhi         15
6  17  manik  42  Delhi         11
8  19  Samir  42  Delhi         15
*****

group name :- Mumbai
*****
   ID  Name  Age  City  Experience
2  13   Aadi  46  Mumbai         11
4  15  Veena  42  Mumbai         17
7  18  vikas  42  Mumbai         15
*****

group name :- Sydney
*****
   ID  Name  Age  City  Experience
0  11   Jack  44  Sydney         19
5  16  Shaunak  42  Sydney         14
9  20  Shobhit  40  Sydney         12
*****

```

In [220...] `df.groupby("City").nunique()`

Out[220...]

	ID	Name	Age	Experience
City				
Delhi	4	4	3	3
Mumbai	3	3	2	3
Sydney	3	3	3	3

In [221...] `df.groupby("Age").nunique()`

Out[221...]

	ID	Name	City	Experience
Age				
40	1	1	1	1
41	1	1	1	1
42	5	5	3	4
44	1	1	1	1
45	1	1	1	1
46	1	1	1	1

In [222...] `# get first row of each group
data.first()`

Out[222...

	ID	Name	Age	Experience
City				
Delhi	12	Riti	41	17
Mumbai	13	Aadi	46	11
Sydney	11	Jack	44	19

In [223...

```
records=data.first()
print("Type of records :- ",type(records))
records
```

Type of records :- <class 'pandas.core.frame.DataFrame'>

Out[223...

	ID	Name	Age	Experience
City				
Delhi	12	Riti	41	17
Mumbai	13	Aadi	46	11
Sydney	11	Jack	44	19

In [224...

```
# get last row of groups
data.last()
```

Out[224...

	ID	Name	Age	Experience
City				
Delhi	19	Samir	42	15
Mumbai	18	vikas	42	15
Sydney	20	Shobhit	40	12

In [225...

```
# get nth row of groups
data.nth(2)
```

Out[225...

	ID	Name	Age	City	Experience
6	17	manik	42	Delhi	11
7	18	vikas	42	Mumbai	15
9	20	Shobhit	40	Sydney	12

In [226...

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    ID          10 non-null    int64
1    Name        10 non-null    object
2    Age         10 non-null    int64
3    City        10 non-null    object
4    Experience  10 non-null    int64
dtypes: int64(3), object(2)
memory usage: 532.0+ bytes
```

```
In [227... # Let perform some aggregate functions
df.groupby("City").agg({"Age": 'mean', "Experience": "max"})
```

```
Out[227...      Age  Experience
City
Delhi  42.500000      17
Mumbai  43.333333      17
Sydney  42.000000      19
```

```
In [228... df.groupby("City").agg({"Age": 'max', "Experience": "min"})
```

```
Out[228...      Age  Experience
City
Delhi   45          11
Mumbai  46          11
Sydney  44          12
```

THANK

YOU

In []: